Week 8. Checkpoint C. Database Implementation

Abstract

This report presents the implementation of a hybrid healthcare sector knowledge graph database. The system integrates structured financial time series data, FDA regulatory documents, and global news coverage for publicly traded healthcare firms using a dual-layer architecture built on PostgreSQL and Memgraph. PostgreSQL functions as the system of record for normalized entity storage and transactional integrity, while Memgraph supports relationship centric analysis through a labeled property graph model and Cypher queries.

1. Introduction

The healthcare sector presents a complex analytical environment in which publicly traded firms operate simultaneously under financial market pressures, regulatory oversight, and evolving media narratives. Regulatory actions such as FDA approvals, recalls, or enforcement notices often produce immediate market responses, and media coverage shapes longer-term investor sentiment. Understanding these dynamics requires a data representation capable of capturing not only entity attributes but also the relationships linking firms, events, and outcomes across time.

This project develops a healthcare sector knowledge graph designed to support competitive intelligence and exploratory financial analysis. The central management problem addressed is how an analyst can efficiently trace the chain from a regulatory event to the companies affected and their subsequent market behavior. Traditional relational queries across isolated tables often require complex joins and key management, making multihop reasoning difficult. A knowledge graph on the other hand, represents relationships and supports intuitive traversal through graph native query languages.

The system integrates multiple heterogeneous data sources, including a curated registry of healthcare companies, FDA regulatory documents collected via focused crawling, Yahoo Finance daily price data,

and news metadata retrieved through the GDELT Document API. These sources are unified within a hybrid architecture that combines PostgreSQL for structured storage and Memgraph for relationship centric analytics.

2. Data Model Design

The relational schema includes normalized entity tables alongside bridge tables that encode relationships in relational form. Core entities include companies, FDA regulatory documents, news articles, trading days, and price points. Relationship tables such as regdoc_mentions_company and news_mentions_company capture document mentions, while company_has_price represents financial linkages across time.

```
.
├── Dockerfile.loader
├── README.md
├── cypher
│   ├── 01_schema.cypher
│   └── 02_queries.cypher
├── data
│   ├── companies.csv
│   ├── financial
│   │   └── yahoo
│   │       ├── ABBV.csv
│   │       ├── AMGN.csv
│   │       ├── BIIB.csv
│   │       .
│   │       .
│   │       .
│   │       └── summary.csv
│   ├── news
│   │   ├── gdelt_articles.jsonl
│   │   └── gdelt_articles_failures.json
│   └── regulatory
│       └── fda_docs.jsonl
├── docker-compose.yml
├── requirements.txt
├── scripts
│   ├── export_to_memgraph.py
│   └── load_postgres.py
└── sql
    ├── 01_schema.sql
    └── 02_queries.sql
```

This design supports efficient joins and structured aggregation in PostgreSQL while mapping cleanly into a graph representation. Core entity tables are:

- company
- regulatory_doc
- news_article
- price_point

Relationship tables:

- regulatory_doc_company
- news_company

This structure allows us to have joins in PostgreSQL while also mapping cleanly into a graph representation.

```
- During export, the ETL pipeline converts:
- companies → (:Company)
- regulatory docs → (:RegulatoryDoc)
- news articles → (:NewsArticle)
- prices → (:PricePoint)
```

Relationships are mapped as:

```
- (:RegulatoryDoc)-[:MENTIONS] → (:Company)
- (:NewsArticle)-[:MENTIONS] → (:Company)
- (:Company)-[:HAS_PRICE] → (:PricePoint)
```

The dual modeling approach also enables consistency across analytical workflows. Queries executed in SQL for validation and aggregation could be replicated or extended in Cypher for multihop reasoning.

3. Implementation Workflow

The system was implemented using a container workflow built using Docker. The environment includes PostgreSQL for structured storage, Memgraph for graph analytics, Memgraph Lab as the interactive UI, and a Python loader container responsible for ETL operations.

Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk read/write	Network I/O	PIDS	Last started	Actions
msds459-healthcare-	-	-	-	0.08%	234.92MB / 30.6	3.01%	10.88MB / 186.7I	108.8MB / 118.7I	48, 6, 1, 11	39 minutes ago	
healthcare_memgr	c402b7e69c3b	memgraph/memgrap	7444:7444 Show all ports (2)	0.06%	100.9MB / 7.65G	1.29%	12.3KB / 14.6MB	24.7MB / 39.2ME	48	1 hour ago	
healthcare_postgre	0735fd3b8ee3	postgres:16	5433:5432	0%	49.07MB / 7.65G	0.63%	10.6MB / 172MB	33.2MB / 10.2ME	6	1 hour ago	
healthcare_loader	516e2ef3cbd3	msds459-healthcare-		0.02%	2.13MB / 7.65GB	0.03%	274KB / 98.3KB	48.4MB / 57.4ME	1	1 hour ago	
healthcare_memgr	30c25bafdaa5	memgraph/lab:latest	3000:3000	0%	82.82MB / 7.65G	1.06%	0B / 0B	2.5MB / 11.9MB	11	39 minutes ago	

The workflow starts with schema creation and structured data loading into PostgreSQL:

```
$docker exec -it healthcare_postgres psql -U kguser -d healthcare_kg -f /sql/01_schema.sql
$docker exec -it healthcare_loader python scripts/load_postgres.py
```

Data sources included a manually curated registry of publicly traded healthcare companies, FDA regulatory documents collected through a focused crawler, Yahoo Finance daily price data, and news metadata retrieved from the GDELT Document API.

Then ETL pipeline exported structured data into Memgraph:

```
$docker exec -it healthcare_loader python scripts/export_to_memgraph.py
```

The export process converts rows into Cypher insertions and reconstructs edges from relational bridge tables.

4. Query Design and Analytical Results

Our SQL queries allow us to validate the relational layer and support structured analytics workflows, for example ranking companies by FDA document mentions, retrieving high-relevance regulatory documents for specific firms and analyzing price behavior across event windows.

```
Export to Memgraph complete.
(base) MM-shm308-22:msds459-healthcare-kg shm308$ docker exec -it healthcare_memgraph mgconsole
mgconsole 1.5
Connected to 'memgraph://127.0.0.1:7687'
Type :help for shell usage
Quit the shell by typing Ctrl-D(eof) or :quit
memgraph> MATCH (n) RETURN labels(n)[0] AS label, count(*) AS cnt;
+-----------------+-----------------+
| label           | cnt             |
+-----------------+-----------------+
| "PricePoint"    | 22608           |
| "NewsArticle"   | 252             |
| "RegulatoryDoc" | 193             |
| "Company"       | 18              |
+-----------------+-----------------+
4 rows in set (round trip in 0.012 sec)
memgraph> MATCH ()-[r]->() RETURN type(r) AS rel, count(*) AS cnt;
+-------------+-------------+
| rel         | cnt         |
+-------------+-------------+
| "MENTIONS"  | 271         |
| "HAS_PRICE" | 22608       |
+-------------+-------------+
2 rows in set (round trip in 0.012 sec)
memgraph> MATCH (d:RegulatoryDoc)-[:MENTIONS]->(c:Company)
       -> RETURN c.ticker, count(d) AS fda_mentions
       -> ORDER BY fda_mentions DESC;
+--------------+--------------+
| c.ticker     | fda_mentions |
+--------------+--------------+
| "BMY"        | 9            |
| "MDT"        | 3            |
| "CVS"        | 2            |
| "BSX"        | 2            |
| "JNJ"        | 2            |
| "MRK"        | 1            |
+--------------+--------------+
6 rows in set (round trip in 0.003 sec)
INFO: Sequential scan will be used on symbol `d` although there is a filter on labels :RegulatoryDoc. Consider creating a label index.
```

Cypher analyses allowed us to search regulatory signals by identifying firms with frequent regulatory mentions. These queries show the system's ability to support realistic competitive intelligence workflows that integrate heterogeneous signals across time.

Cypher editor | Graph Style editor | Parameters

```
MATCH (d:RegulatoryDoc)-[:MENTIONS]->(c:Company)
RETURN c.ticker, c.name, count(d) AS fda_mentions
ORDER BY fda_mentions DESC;
```

Share query | Run query

Query successful — 7 ms — 6 — 0 — 0

Data results | Graph results unavailable | Summary — Copy all | Download | Fullscreen

#	c.ticker	c.name	fda_mentions
1	BMY	Bristol Myers Squibb Company	9
2	MDT	Medtronic plc	3
3	CVS	CVS Health Corporation	2
4	BSX	Boston Scientific Corporation	2
5	JNJ	Johnson & Johnson	2
6	MRK	Merck & Co., Inc.	1

5. Challenges and Limitations

I had three main implementation challenges during development. The most challenging one was GDELT API rate limiting (HTTP 429 responses), which restricted the volume of news data that could be collected in a single pass. While this initially slowed ingestion, the issue was solved through a combination of exponential backoff, request throttling, and segmented queries across smaller time windows.

Secondly, matching company mentions across FDA documents, news metadata, and structured registries required a lot of manual validation. For the final project, I plan to explore more robust solutions such as a transformer based entity recognition pipeline to improve both recall and precision.

Finally, I encountered some initial challenges while setting up the Docker environment, around container networking and port conflicts. Local services were already using default ports (e.g., PostgreSQL on 5432). These issues were resolved by reconfiguring port mappings.

6. Reflection

PostgreSQL provided strong guarantees for structured storage and validation workflows, while Memgraph enabled intuitive exploration of relationships that would otherwise require complex joins. The system showed how relational and graph paradigms can complement rather than replace each other. The knowledge graph successfully addresses the original management question: enabling analysts to trace regulatory events to affected companies and downstream market behavior.

Future work would include the model with richer NLP-based extraction, improved news ingestion pipelines, and graph-based predictive modeling for event-driven price forecasting.

References

- Code Repository: https://github.com/mehdi114455/MSDS-459-Week-08.git
- Docker Documentation: https://docs.docker.com/
- Memegraph: https://memgraph.com/docs
- Chatgpt was used some of the coding aspects like fixing the GDELT API rate limiting issue.